Repair cost under a single list edit

A measurement of DCG difference for `mergeSort`

Fumola

7 September 2026

Abstract

Fumola has no repair algorithm. That is deliberate while the specification for expected behaviour is still being written, and it means repair cost cannot be measured directly. What can be measured is the difference a small edit makes to the demanded computation graph, which bounds what any repair would have to do: nodes both runs name and agree on are what a repair gets for free, and everything else it must account for. This report samples that difference for the two phases of `mergeSort` under a single list removal. The list-into-tree phase is stable — the number of changed nodes stays near constant from $n = 16$ to $n = 1000$ while the graph grows $62\times$, so the changed fraction falls from 0.171 to 0.0035. The merge network is not: its cost is heavier, far more variable, and tracks how many merge thunks a single symbol names.

1 What is measured

Two runs are compared, identical but for one removed list cell. Each run is reset first, so its history is its own. The comparison is `A.Diff.nodeValsDiff`, which keys nodes by pointer and ignores edge ids and meta times because neither affects whether a cached result can be reused. It yields four numbers:

onlyLeft	the left run named this node; the right run did not
equal	both named it and agree on its content
notEqual	both named it and disagree
onlyRight	the right run named it; the left did not

Write *changed* for onlyLeft + onlyRight + notEqual and *diffFraction* for changed over the larger of the two runs' node counts. A repair that had to rebuild every changed node and could reuse every equal one would do work proportional to *changed*.

This is an upper bound on the useful measurement, not the measurement itself. A real repair may touch fewer nodes than differ, and it may touch nodes that do not differ in order to discover that they do not. The four numbers say what *could* be reused, which is the part the naming decisions control.

Sampling. Each sample draws a fresh input seed and a fresh removal position, both from the seeded generator rather than swept, so no structure of a sampling grid can be mistaken for structure in the answer. Removal positions fall in $2 \dots n$: the head is not addressable, because the edit cursor starts one cell in (issue #77).

2 The list-into-tree phase scales

n	samples	nodes	changed	mean	median	p90	max	$\log_2 n/n$	ratio
16	200	80	13.7	0.1708	0.1625	0.225	0.275	0.25	0.68
32	200	160	13.7	0.0858	0.0813	0.1125	0.1688	0.1563	0.55
64	150	320	14.3	0.0446	0.0375	0.0719	0.1125	0.0938	0.48
100	150	500	15.2	0.0304	0.026	0.052	0.07	0.0664	0.46
250	100	1,250	15.7	0.0126	0.0104	0.0208	0.0312	0.0319	0.4
500	60	2,500	17.6	0.007	0.0052	0.0132	0.03	0.0179	0.39
1,000	40	5,000	17.5	0.0035	0.0026	0.0074	0.015	0.01	0.35

Table 1: Repair cost for list-into-tree, by input size. *nodes* and *changed* are means; the four *diffFraction* columns summarise its distribution; *ratio* is the mean against $\log_2 n/n$.

Two things stand out. The changed fraction falls by a factor of 49 as n grows $62\times$, and it stays *below* $\log_2 n/n$ at every size, with the ratio itself improving from 0.68 to 0.35. Figure 1 shows this against that reference.

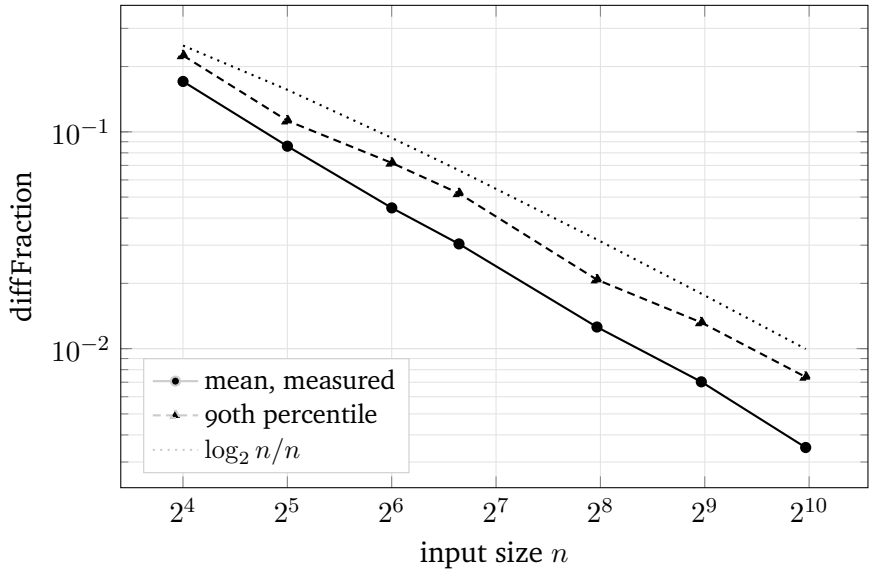

Figure 1: The changed fraction of the graph, against $\log_2 n/n$. Even the 90th percentile stays under the reference from $n = 64$ up.

The reason is Figure 2: the *absolute* number of changed nodes barely moves. It goes from 13.7 at $n = 16$ to 17.5 at $n = 1000$ — a $1.28\times$ increase across a $62\times$ increase in size. ONLYLEFT is exactly 4.00 at every size measured, and all of the growth is in NOTEQUAL, which rises from 9.7 to 13.5. Divided by $\log_2 n$ the count *falls*, from 3.42 to 1.76, so over this range the phase is behaving better than $O(\log n)$ per edit rather than merely as well.

3 The two phases differ

Both phases can be separated from one pair of runs, because each is built inside its own space, and the space is the pointer's prefix. A node of the first reads `inputTree(3-element)`; a node of the second reads `lazyMergeSort(merge-11)(3)`. Table 2 compares them at $n = 16$ over 12 seeds and all 15 addressable positions.

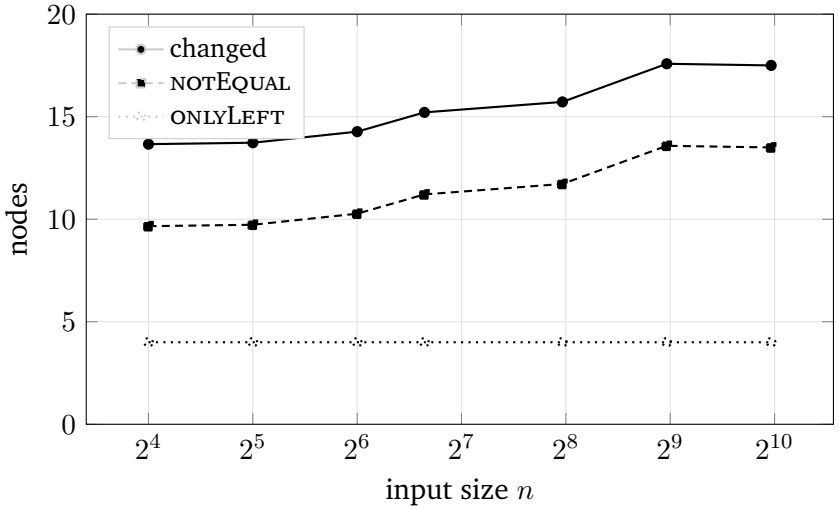

Figure 2: Changed nodes per edit, on a log size axis. Near constant: onlyLeft is 4.00 throughout, and onlyRight is 0 at every sample.

phase	samples	nodes	changed	mean	median	p90	max
list	180	20	5	0.249	0.25	0.25	0.25
tree	180	62	11	0.177	0.161	0.274	0.306
merge	180	86.2	24	0.278	0.221	0.59	0.758

Table 2: The three spaces at $n = 16$. The merge network is costlier on average and much heavier in the tail.

The averages differ by less than a factor of two, but the tails do not. The merge network's 90th percentile is 0.590 against the tree's 0.274, and its worst case 0.758 against 0.306.

4 Only one of the two phases scales

Both phases can be sampled together, at the cost of full-demand runs, which is why this reaches only $n = 100$ where the tree phase alone reaches $n = 1000$. Sample counts are small — 40, 30, 20 and 12 — and the merge network's distribution is heavy-tailed, so read Table 3 as a trend and not as an estimate.

	tree			merge network			
n	changed	mean	ratio	changed	mean	p90	ratio
16	11.5	0.1851	0.74	22.5	0.254	0.5294	1.02
32	10.5	0.0836	0.54	29.4	0.1383	0.2679	0.89
64	11.8	0.0465	0.5	52.2	0.1022	0.3922	1.09
100	10.8	0.0272	0.41	29.6	0.0309	0.0444	0.47

Table 3: Both phases, from the same runs. *ratio* is the mean diffFraction against $\log_2 n/n$. Samples per size: 40, 30, 20, 12.

The tree phase's ratio improves steadily, $0.74 \to 0.54 \to 0.50 \to 0.41$, and its changed count stays near 11 at every size. The merge network's ratio sits near 1 for the first three sizes — 1.02, 0.89, 1.09 — and its changed count grows with n rather than staying flat. The drop to 0.47 at

$n = 100$ comes from twelve samples and should not be read as the trend turning; at that sample size an expensive position simply may not have been drawn.

What is solid at every size, because it does not depend on the small samples, is the ordering: the merge network changes two to four times as many nodes as the tree does, and its 90th percentile runs three to five times the tree's.

5 What the merge network's names cost

A merge thunk is currently named for the input-tree node whose stream it is built from — the S in `merge-`S. That makes a symbol's influence proportional to how many streams descend from its tree node, which is largest at the root. Counting over 12 seeds at $n = 16$, a symbol names 5.2 merge nodes on average and as many as 12.8, against $\log_2 16 = 4$: the maximum is $3.2\times$ what an evenly spread naming would give.

Figure 3 plots the consequence. Across the 15 positions, the correlation between a position's stream-owner count and the merge network's diff Fraction is $+0.67$.

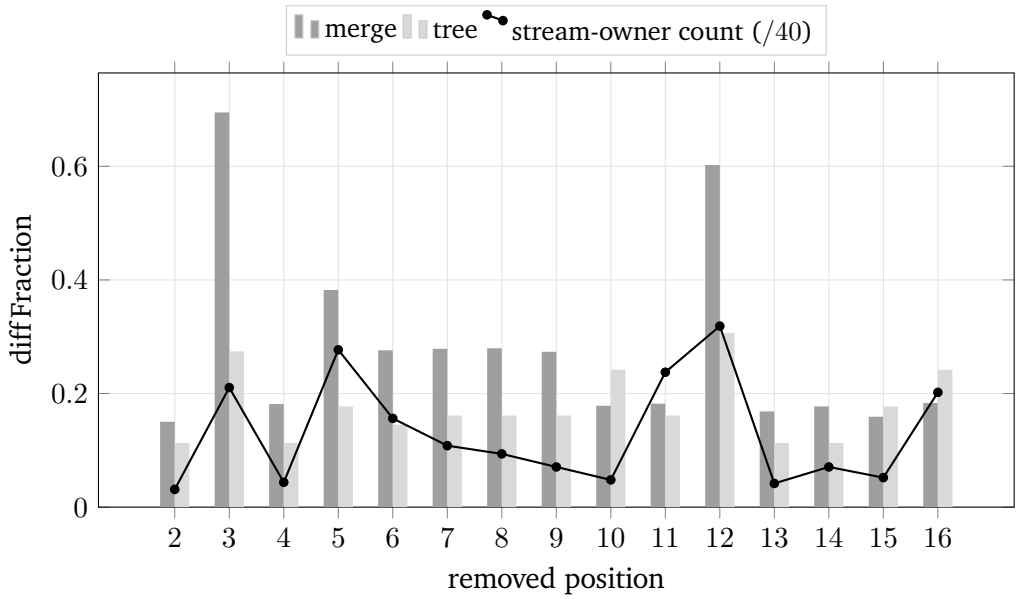

Figure 3: Cost by removed position at $n = 16$. Positions 3 and 12 are the expensive ones; position 3 is the symbol whose level roots every tree over positional symbols (issue #72).

Removing cell 3 rebuilds 69.5% of the merge network while changing only 27.4% of the tree. Cell 3's symbol has the highest level of the first several hundred, so it roots every tree built over positional symbols, and its identity propagates into every stream built from it. Two effects compound: being a heavy stream owner, and sitting high in the tree. Positions 11 and 16 own many streams but sit low, and stay cheap at 0.182 and 0.183 — which is why the correlation is 0.67 and not higher.

This supports a renaming already proposed: name each merge thunk for the *pair of input cells whose comparison produced it*, rather than for the tree node the stream came from. Under that scheme the root symbol should name $O(\log n)$ nodes like any other. This report does not test that scheme, which is not implemented; it measures the one that is, and finds the concentration the proposal predicts.

6 Limits

- **No repair algorithm exists**, so nothing here measures repair. It measures what a repair could reuse. A repair that walks nodes to discover they are unchanged pays costs this does not capture.
- **One edit shape.** Only removal, and only one cell at a time. Insertion and replacement are not sampled, and neither are edit *sequences* — K successive edits, where reuse across edits is the thing that matters.
- **The head is not addressable**, so position 1 is absent from every sample (issue #77).
- **Merge-network scaling is under-powered.** It reaches only $n = 100$, on 40, 30, 20 and 12 samples, because it needs full-demand runs where the tree phase alone can skip layout. The ordering between the phases is solid at every size; the shape of the merge network's scaling curve is not established by this data.
- **Positional symbols throughout.** Every input here is keyed $1 \ldots n$, which is one fixed realisation of the level function rather than a sample of them (issue #72). Entropy-carrying symbols would change which positions are expensive, though not that some are.